

Лабораторная работа №7

Дискретное логарифмирование в конечном поле. ρ -метод Полларда

Кюнкриков Даниил Саналович (НПИМд-01-24, 1132249574)

Российский университет дружбы народов (РУДН), Москва

- Кюнкриков Даниил Саналович
- студент уч. группы НПИМд-01-24
- Российский университет дружбы народов
- 1132249574@pfur.ru
- репозиторий: https://github.com/DanzanK/2025-2026_math-sec/tree/main

Цель работы: реализовать ρ -метод Полларда для решения $a^x \equiv b \pmod{p}$.

Задачи:

- описать постановку DLP в $\mathbb{F}_p$;
- реализовать поиск коллизии (Floyd: «черепаха–заяц»);
- восстановить x через решение линейного сравнения по модулю r ;
- проверить на контрольном примере.

Найти x , такое что

$$a^x \equiv b \pmod{p}, \quad a, b \in \mathbb{F}_p^*.$$

Если r — порядок элемента a , то решение рассматривается по модулю r .

- строим последовательность $c_{i+1} = f(c_i)$ в группе;
- поддерживаем представление $c \equiv a^\alpha b^\beta$;
- ищем коллизию $c = d$ методом Флойда:
 - $c \leftarrow f(c)$ (1 шаг)
 - $d \leftarrow f(f(d))$ (2 шага)

При коллизии:

$$(\beta_1 - \beta_2)x \equiv (\alpha_2 - \alpha_1) \pmod{r}.$$

Решение линейного сравнения

После коллизии решаем:

$$Ax \equiv \Delta \pmod{r},$$

где $A = (\beta_1 - \beta_2) \bmod r$, $\Delta = (\alpha_2 - \alpha_1) \bmod r$.

Используем расширенный алгоритм Евклида:

- $g = \gcd(A, r)$
- если $g \nmid \Delta \rightarrow$ перезапуск
- иначе $x \equiv A^{-1} \Delta \pmod{r/g}$

```
# Расширенный алгоритм Евклида
def egcd(a: int, b: int):
    if b == 0:
        return (a, 1, 0)
    g, y1, x1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)
def rho_dlog(p: int, a: int, b: int, r: int):
    # строим псевдослучайную последовательность в группе и ищем коллизию c == d,
    # Случайная инициализация логарифмов (u, v) в диапазоне [0, r-1]
    u = secrets.randbelow(r)
    v = secrets.randbelow(r)
    # Стартовая точка:
    c = (pow(a, u, p) * pow(b, v, p)) % p
    # Для поиска коллизий используем "черепаха-заяц" (Floyd):
    d = c
    alpha1, beta1, alpha2, beta2 = u, v, u, v
    # Простое разбиение множества состояний на 2 части
    half = p // 2

    def step(x: int):
        if x < half:
            return (a * x) % p
        else:
            return (b * x) % p

    # Основной цикл: идём, пока не найдём коллизию c == d
    while True:
        c = step(c)
        alpha1 = (alpha1 + (1 if c < half else 0)) % r
        beta1 = (beta1 + (1 if c >= half else 0)) % r
        d = step(d)
        d = step(d)
        alpha2 = (alpha2 + 2 * (1 if d < half else 0)) % r
        beta2 = (beta2 + 2 * (1 if d >= half else 0)) % r

        if c == d:
            break
        A = (beta1 - beta2) % r
        delta = (alpha2 - alpha1) % r
        # Решаем A*x ≡ delta (mod r)
        # Если g = gcd(A, r) не делит delta - решения нет (Invalid problem)
        g, xcoef, _ = egcd(A, r)
        if delta % g != 0:
            return None
        return (xcoef * (delta // g)) % r
```

Рисунок 1: Программная реализация метода Полларда по разложению чисел на множители

Результат

Параметры: $p = 107$, $a = 10$, $b = 64$, $r = 53$.

```
PS C:\Users\Danzan\Desktop\MOZiiIB\лр> & C:/ProgramData/anaconda3/python.exe c:/
anzan/Desktop/MOZiiIB/лр/ЛР7/LR7.py
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 53
Invalid problem
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 53
x = 2
Введите p, a, b, r через пробел, Выход для того чтобы выйти
107 10 64 13
x = 12
Введите p, a, b, r через пробел, Выход для того чтобы выйти
█
```

- Реализован ρ -метод Полларда для дискретного логарифмирования в $\mathbb{F}_p$.
- Алгоритм является вероятностным; при неудаче выполняется перезапуск.
- На контрольном примере получен корректный результат.